



How to Monitor & Log Spark Jobs in Databricks

Debugging Spark jobs in Databricks is crucial for maintaining reliable data pipelines. Let's explore how to **monitor, log, and debug errors effectively** using Databricks tools.



Abhishek Agarwal
Data Engineer



Why Monitor Spark Jobs?

Spark jobs can fail due to:

- ✓ Data Skew
- ✓ Memory Overflows
- ✓ Slow Shuffles
- ✓ Long-running Queries

Proactively monitoring Spark jobs helps prevent failures & optimize performance!



Monitoring Jobs in Spark UI

The Databricks Spark UI provides deep insights into execution plans & job performance.

✓ How to Access Spark UI:

- 1 Open your **Databricks notebook**
- 2 Run a job and click on the **View Spark UI** link
- 3 Navigate through **Jobs, Stages, Tasks, and Executors**

Use the UI to detect long-running tasks, bottlenecks, and memory issues!

Checking Logs in Databricks UI

Databricks provides detailed logs for each job execution.

✓ How to View Logs:

- 1 Open the **Databricks Jobs UI**
- 2 Click on the **Job Run ID**
- 3 Expand the **Output** section to check error messages
- 4 Download the full log if needed

Use logs to track errors & debug failed tasks!



Understanding Job & Stage Execution

Monitor Spark job execution flow in Databricks!

- ✓ **Jobs Tab:** Shows all running and completed Spark jobs
- ✓ **Stages Tab:** Displays different processing stages
- ✓ **Tasks Tab:** Drills down into specific tasks per stage
- ✓ **Executors Tab:** Shows memory, CPU, and resource utilization

Analyze long-running tasks & optimize shuffle partitions!



Common Errors & Debugging Steps

Identify common Spark job errors & their solutions.

✓ Memory Errors (OutOfMemoryError)

- Increase **executor memory**
- Optimize **data partitions**
- Use **broadcast joins** for small tables

✓ Data Skew Issues

- Check **skewed partitions in Spark UI**
- Use **Salting technique** for even distribution

✓ Slow Queries & Shuffles

- Reduce **shuffle partitions** dynamically
- Use **Adaptive Query Execution (AQE)**

Debug faster with structured logging & performance tuning!



Logging Errors with dbutils

Log messages & errors using dbutils for debugging.

✓ **Example: Logging in Databricks Notebook**

```
import logging
```

```
logger = logging.getLogger("databricks")  
logger.setLevel(logging.INFO)
```

try:

```
df = spark.read.parquet("s3://invalid-path/")  
df.show()  
except Exception as e:  
    logger.error(f"Error reading data: {e}")
```

Store logs for future analysis & troubleshooting!



Using Cluster Event Logs

Monitor Databricks cluster logs for deeper insights!

✓ Types of logs available:

- **Driver Logs:** Application logs from the main driver node
- **Executor Logs:** Logs from distributed worker nodes
- **Event Logs:** Detailed execution timeline

✓ How to Access Logs:

- 1 Navigate to **Clusters** in Databricks
- 2 Select your running **cluster**
- 3 Click on the **Logs** tab

Use logs to debug distributed processing issues!

Enabling Structured Logging in Databricks Jobs

Save logs to cloud storage for long-term analysis

✓ **Example: Write logs to ADLS/S3/GCS**

```
log_df = spark.createDataFrame([("Error reading file",)],  
["log"])  
log_df.write.mode("append").parquet("s3://my-  
bucket/logs/")
```

Centralized logging makes debugging easier!



Setting Up Alerts for Job Failures

Automatically get notified when Spark jobs fail!

✓ Steps to Configure Alerts:

- 1 Go to **Databricks Jobs**
- 2 Select the job & open the **Alerts** tab
- 3 Set **Failure Alerts** via Email, Slack, or Webhooks

Instant notifications help resolve failures faster!



Final Takeaways – Debug Smarter!

- ✓ **Use the Spark UI** to monitor job execution
- ✓ **Check Databricks logs** for error tracking
- ✓ **Enable structured logging** for long-term analysis
- ✓ **Optimize queries** to avoid memory & performance issues
- ✓ **Set up alerts** to catch failures in real-time

Debug faster, optimize smarter, and keep your Spark jobs running smoothly in Databricks!



**Follow for more
content like this**



Abhishek Agarwal

Azure Data Engineer



Abhishek Agarwal | Azure Data Engineer